

NATIONAL UNIVERSITY OF SINGAPORE

EE4705: Human Robot Interaction

Project 1.2: An AI-bot in World Model (20%)

I. Instruction

1. This project is a joint work to show your collaborative effort, and at the same time, each individual can show his/her talent for individual portions as your choices and declaration. Students are required to work together to complete it with 3 students (2 students in special case) as a group.
2. Zip all files into one compressed file. Name your file as project_1.2 with your group index and submit it into the folder Student Submission: Project 1.2 under Canvas.
Example: if your group index is 1, the file name must be project_1.2_group_1.zip.
3. Put your names and matriculation numbers on the cover page. There should only be one **combined report** submitted per group. Recommended pages: 8–15 pages, 1.5-line spacing, 12-point “Times New Roman” font, 1-inch margins. The zipped file must include: (a) the report in PDF; (b) all your source code (Python/ROS2 packages) with a README explaining how to run it; (c) your map/world/waypoint files; and (d) a 3–5 minute demo video (or an unlisted YouTube/Google Drive link in the report). Include proper citations for all sources.
4. The submission deadline is on **27th Sep 2026**; any submissions after that will be subject to the late submission policy.
5. Group members can choose how to split the tasks amongst themselves; however, the tasks should be split evenly to ensure that each member's contribution can be adequately appraised. Each student must have their contribution and effort clearly stated for their report section, and contributions must also be marked in the source files. Identical reports will be subject to penalty and/or disciplinary actions of the university.
6. You are encouraged to purchase physical robot hardware and complete the project on a real robot.
7. Use of AI coding assistants (e.g., ChatGPT, Claude, GitHub Copilot, Cursor) is allowed and encouraged in this project to accelerate development (“vibe coding”).
8. Any queries, feel free to contact GAs:
 - Mr. Huang Dong, dong.huang@u.nus.edu, and
 - Mr. Ying Zhuohang, E1554788@u.nus.edu.

II. Objective

Multimodal large language models (MLLMs / VLMs) such as GPT-5.5, Qwen-VL, Gemini and LLaVA give robots an unprecedented ability to see, understand and talk about their surroundings. In this project you will give the VLM system a body: a simulated mobile robot that you can command in natural language, that looks around through its camera, tells you what it sees, and physically acts on your follow-up instructions. This closed loop of command → perception → action → command is the essence of modern human-robot interaction. The objectives of this project are:

1. To familiarize with a modern robot simulation stack (ROS2 + Gazebo, or an alternative simulator of your choice).
2. To familiarize with mobile robot navigation: mapping (SLAM), localization and waypoint navigation (Nav2).
3. To learn how to connect a Vision-Language Model to a robot's camera and control loop through an API.
4. To learn structured-output prompting (function calling / JSON mode) for translating free-form natural language into robot commands.
5. To design a multi-turn human-robot interaction workflow that combines language, visual perception and physical action.
6. To practice systematic evaluation of an embodied AI system: task success rate, grounding accuracy, latency and cost.
7. To provide practical experience in problem-finding and problem-solving when integrating LLMs/VLMs with robots.

The reference interaction. Throughout this document we use the following interaction as the reference scenario your final system must support. The rooms of the simulated house are numbered on the map (Room 1, Room 2, ...), and several everyday objects (e.g., a cup, a ball, a box) are placed in the rooms:

```
User:   Go to Room 3 and have a look. What can you see?
Robot:  (navigates to Room 3 autonomously) I have arrived in Room 3.
        I can see a table with a red cup on it, and a chair next to the table.
User:   Move to the cup.
Robot:  (finds the cup in its camera view, drives towards it, stops nearby)
        I am now next to the cup. Anything else?
User:   Is there anything on the floor?
Robot:  I can see a blue ball on the floor near the chair.
```

Note: each group only needs to submit one completed system; members' contributions need to be marked in the report and in the source files.

III. Tasks

The project consists of five tasks. Figure 1 shows the overall architecture that the five tasks build up together. You may deviate from this architecture if you can justify your design in the report.

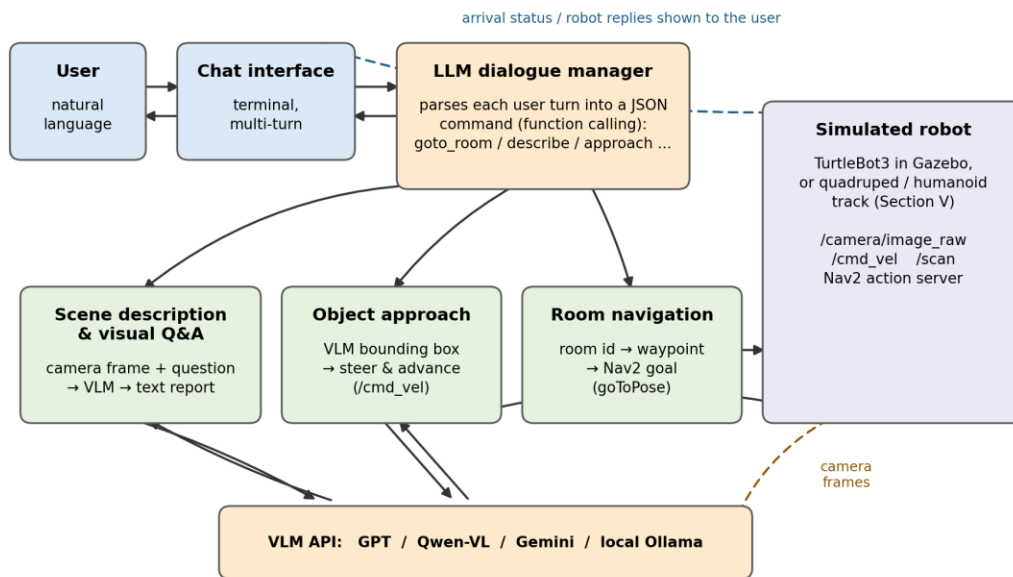


Fig. 1. Reference architecture of the VLM-powered interactive robot assistant.

Task 1: Set Up the Robot Simulation Platform (ALL)

- i. Conduct a short literature review on Vision-Language Models for robotics: what VLMs are and how they differ from text-only LLMs; and how recent systems connect LLMs/VLMs to robots (e.g., SayCan, Code as Policies, ROSGPT, NASA JPL ROSA, LLaVA, Qwen-VL). Keep it within 2 pages and cite properly.
- ii. Choose your robot morphology and simulator from Section V (wheeled robot, quadruped, or humanoid — the wheeled TurtleBot3 track is the recommended default) and set up the environment. Drive the robot around with keyboard teleoperation and verify that you can view the robot's camera images.

- iii. Prepare a multi-room world. For the default track, use the TurtleBot3 “house” world in Gazebo. Assign numbers to at least 4 rooms/areas (Room 1, Room 2, ...) and place at least 3 distinct everyday objects (e.g., a cup or can, a ball, a box) in different rooms. Include a labelled floor plan (your SLAM map with room numbers) in the report.
- iv. Write down the problems you encountered during environment setup, the solutions, and your experiences.

Task 2: Natural-Language Commanded Room Navigation (Student A)

- i. Design the command interface between the manager and the robot: a small JSON command schema (e.g., goto_room / describe / approach / stop / chat) and an LLM-based parser that converts a free-form user utterance into one JSON command. The parser must be robust to paraphrases (“Go to Room 3”, “could you check room three please”, “head over to the third room”) and must reject or ask for clarification on invalid requests (“go to Room 99”). Document your prompt design in the report.
- ii. Build the map and the room waypoint table: run SLAM to map your world, save the map, and record one goal pose (x, y, yaw) per numbered room in a configuration file. Implement navigation so that a parsed goto_room command sends the corresponding goal to the Nav2 stack (or your platform's waypoint controller) and reports success/failure back to the user in natural language.
- iii. Implement the multi-turn chat interface in the terminal: the user confirms input with the ENTER key, the robot executes and replies, and the conversation continues. The chat history must be maintained so that follow-up turns can refer to earlier ones.
- iv. Evaluate the command parser on at least 20 test utterances (including at least 5 paraphrases and 5 invalid/out-of-scope requests). Report the parsing accuracy and analyze the failure cases.

Task 3: VLM Scene Understanding and Reporting (Student B)

- i. When the robot arrives in a room (or on a describe command), capture one or more camera frames — optionally rotating in place to scan the room — send them to a VLM together with a suitable prompt, and have the robot report concisely what it sees (“I can see a table with a red cup on it...”). The report should reach the user through the chat interface as the robot's reply.
- ii. Support follow-up visual question answering about the current view, e.g., “Is there anything on the floor?”, “What colour is the cup?”, “How many chairs are there?”.
- iii. Compare at least two VLM services (e.g., GPT-4o-mini vs. Qwen-VL vs. a local model via Ollama) on at least 10 scenes from your world: correctness of the description (objects correctly reported / missed / hallucinated), response latency, and cost per call. Present the comparison in a table and discuss which model you chose for the final system and why.

Task 4: Language-Directed Object Approach (Student C)

- i. Implement visual grounding: given a command such as “move to the cup”, query the VLM for the target object's bounding box in the current camera image (or use an open-vocabulary detector). Handle the object-not-visible case with a simple search behaviour (e.g., rotate in place and re-query, report failure after a full turn).
- ii. Implement the approach controller: steer so that the bounding box centre moves to the image centre, drive forward, and stop when the robot is close to the object (e.g., using the bounding-box size and/or the laser scanner distance). The robot must stop safely before collision and confirm to the user (“I am now next to the cup”).
- iii. Evaluate the grounding and approach on at least 10 trials with different objects and different starting poses: grounding accuracy (did the VLM select the right object), approach success rate, final distance to the object, and failure analysis.

Task 5: Full-System Integration, Evaluation and Demonstration (ALL)

- i. Integrate Tasks 2–4 into one continuous multi-turn HRI session that supports the reference interaction in Section II: the manager decides, for every user turn, which capability (navigate / describe / approach / chat) to invoke, and the robot's replies close the loop back to the user.
- ii. Perform a systematic end-to-end evaluation: at least 20 randomized trials varying the target room, the target object and the phrasing of the commands. Report per-stage success rates (parsing, navigation, description, approach), end-to-end success rate, average latency of each stage, and total API cost. State clearly how you define “success” for each stage.
- iii. Record a 3–5 minute demo video of one uncut session covering at least 2 rooms and 2 object approaches, including one case where the object is not initially visible.
- iv. Discussion in the report: main failure modes and their causes, limitations of the VLM (hallucination, latency, spatial reasoning), safety considerations, and what would change if the system were deployed on a real robot.

Optional bonus (up to 5 extra marks, capped at 100): voice interaction; a GUI chat window; a second robot morphology running the same manager; or a “no room numbers” mode where the robot finds “the room with a bed” using vision alone.

IV. Score Distributions

In this project, we will thoroughly assess and assign scores to students based on their performance in three key areas: innovation, usefulness, and teamwork. Specifically, we will evaluate how creatively they approach problem-solving and propose novel ideas (innovation), how practical and applicable their solutions are in addressing real-world challenges (usefulness), and how effectively they collaborate, communicate, and contribute as part of a group (teamwork). Each of these aspects will be carefully considered to provide a comprehensive evaluation of the students' overall contribution to the project.

The distribution of the scores for each task is listed below.

Groups with 3 members

Task	Score Distribution	Name
1	20%	ALL
2	60%	Student A
3	60%	Student B
4	60%	Student C
5	20%	ALL

V. Robot Platforms and VLM Services for Your Consideration

You are free to choose the robot morphology and the simulator. Any combination is acceptable as long as: (a) the simulated robot has an onboard RGB camera; (b) it can be commanded to move (velocity commands and/or waypoint navigation); and (c) the world contains several rooms or zones that can be numbered. The following tracks are verified to work on ordinary laptops without a dedicated GPU (the VLM always runs in the cloud via API). Track 1 is the recommended default and is the one used in the Guiding Example.

Track	Robot / Simulator	Requirements	Key open-source resources
1 (default)	Wheeled TurtleBot3 (waffle_pi), ROS2 Humble + Gazebo + Nav2	Ubuntu 22.04 (native, WSL2 on Windows 11, or VM)	turtlebot3_simulations (ROBOTIS), TurtleBot3 e-Manual, Nav2
2	Wheeled TurtleBot3 Burger in Webots + webots_ros2 (ships lidar-only — adding a Camera node to the robot's extensionSlot is part of this track)	Webots GUI runs natively on Windows/macOS; ROS2 side in WSL2/Linux	webots_ros2 (Cyberbotics)
3	Quadruped (“robot dog”): CHAMP or Unitree Go2 in Gazebo, velocity-commanded walking	Ubuntu 22.04/24.04; CPU only	champ (ros2 branch), unitree-go2-ros2, unitree_go2_ros2 (Jazzy)
4	Humanoid: NAO in Webots (built-in motion files) or Pepper in qiBullet (pure Python)	Windows/macOS/Linux native; no ROS required	Webots NAO, qiBullet
5 (advanced)	Humanoid Unitree G1 walking with a pre-trained RL policy in MuJoCo (pure Python)	Any OS; CPU only; no ROS	unitree_rl_gym (official, pre-trained G1/Go2/H1 policies)
Fallback	PyRoboSim 2D semantic world (rooms/objects built in)	Any OS; pip install; use rendered screenshots for the VLM	pyrobosim — discuss with the GA before choosing this track

Notes: for Tracks 3–5 the locomotion is already solved by the cited packages (velocity-commanded walking); your work remains the same perception/action layer. The quadruped simulations in Track 3 ship with lidar but no camera — attaching a standard Gazebo camera sensor to the robot's URDF/xacro is a well-scoped part of that track (ask your AI assistant; it is ~15 lines of XML). Track 5 additionally needs pip install mujoco and CPU PyTorch. For Track 4, qiBullet is unmaintained: use a Python 3.10/3.11 virtual environment and note that the first run asks you to accept the robot-mesh licence. For robots without Nav2, replace “navigation goal” with a simple waypoint-following controller on velocity commands, and room waypoints with known coordinates — the tasks and grading are unchanged. GitHub links for all resources are given in the References.

VLM/LLM services (choose at least two for the comparison in Task 3):

Service	Model (example)	Cost / free tier	Note
OpenAI	gpt-5-mini / gpt-5-nano (gpt-4o-mini also still works)	paid, very cheap ($\approx$ US\$1–2 for the whole project at image detail “low”)	simplest, good all-round
Alibaba Cloud Model Studio (Singapore region)	qwen3-vl-plus / qwen3-vl-flash	free quota for new users ($\approx$ 1M tokens per model, 90 days); enable “free quota only” to avoid billing	best choice for grounding (bounding boxes)
Google AI Studio	gemini-2.5-flash or newer Flash	free tier, rate-limited — check your live limits in AI Studio	good free option; uses its own google-genai SDK
Ollama (local)	qwen2.5vl:3b / llava	free, runs on CPU	offline fallback; slow — use for snapshots, not control loops

VI. Guiding Example

This section walks through the recommended Track 1 (TurtleBot3 + ROS2 Humble + Gazebo). It gives you a working skeleton for every task; you are expected to extend, restructure and improve it. Where a command or API differs on your machine, consult the cited official documentation — and let your AI coding assistant help you debug.

1. Configure the working environment

You need Ubuntu 22.04. We use ROS2 Humble (LTS, supported until May 2027) because the TurtleBot3 Gazebo simulation targets it; when consulting the TurtleBot3 e-Manual, always select the Humble tab — the Jazzy tab shows different commands for a different Gazebo. On Windows 11, WSL2 with WSLg runs the Gazebo GUI out of the box (run `wsl --update` first; if Gazebo shows a black window, put `export LIBGL_ALWAYS_SOFTWARE=1` in `~/.bashrc`). A virtual machine or dual boot also works. On Apple-Silicon Macs, use a Ubuntu 22.04 ARM virtual machine (e.g., UTM) or consider Track 2/4/5. Install ROS2 Humble following the official guide (docs.ros.org), then install the TurtleBot3 and navigation packages:

```
sudo apt update
sudo apt install ros-humble-desktop
sudo apt install 'ros-humble-turtlebot3*' ros-humble-nav2-bringup \
                ros-humble-nav2-simple-commander python3-pip
pip3 install openai opencv-python
echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc
echo 'export TURTLEBOT3_MODEL=waffle_pi' >> ~/.bashrc
source ~/.bashrc
```

Important: use the `waffle_pi` model — it carries a camera; the `burger` model does not. Do not mix conda environments with ROS2; use the system Python.

2. Launch the house world and check the camera

```
# Terminal 1: the multi-room house world
ros2 launch turtlebot3_gazebo turtlebot3_house.launch.py

# Terminal 2: drive around with the keyboard
ros2 run turtlebot3_teleop teleop_keyboard

# Terminal 3: check and view the on-board camera
ros2 topic hz /camera/image_raw
ros2 run rqt_image_view rqt_image_view
```

Note: the first launch of the house world downloads models and may sit at a black screen for several minutes — do not kill it. The house contains several connected rooms — decide your Room 1...N numbering now. Insert extra objects (e.g., a can, a ball, cardboard boxes) into different rooms using the Gazebo GUI (Insert panel), and save your modified world file so that your setup is reproducible.

3. Build the map and set up navigation

```
# SLAM (drive around with teleop until the whole house is mapped)
ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=True
# in a NEW terminal, while cartographer is still running:
ros2 run nav2_map_server map_saver_cli -f ~/house_map

# Navigation with the saved map
ros2 launch turtlebot3_navigation2 navigation2.launch.py \
    use_sim_time:=True map:=$HOME/house_map.yaml
```

In RViz, use “2D Pose Estimate” to localize the robot, then “Nav2 Goal” to test navigation. Read the goal poses of your numbered rooms from RViz (or the map) and record them in a small configuration file:

```
# rooms.yaml (poses read from your own map – these numbers are examples)
rooms:
  1: {x: 0.0, y: 1.8, yaw: 1.57}    # living room
  2: {x: 3.5, y: 1.0, yaw: 0.0}    # kitchen
  3: {x: 6.2, y: -0.5, yaw: -1.57} # bedroom
  4: {x: -4.8, y: 2.6, yaw: 3.14}  # study
```

Sending a goal from Python uses the Nav2 Simple Commander API:

```
import math, yaml, rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator
from geometry_msgs.msg import PoseStamped

def make_pose(nav, x, y, yaw):
    p = PoseStamped()
    p.header.frame_id = 'map'
    p.header.stamp = nav.get_clock().now().to_msg()
    p.pose.position.x, p.pose.position.y = x, y
    p.pose.orientation.z = math.sin(yaw / 2)
    p.pose.orientation.w = math.cos(yaw / 2)
    return p

rclpy.init()
nav = BasicNavigator()
nav.waitUntilNav2Active()
rooms = yaml.safe_load(open('rooms.yaml'))['rooms']

def goto_room(n):
    r = rooms[n]
    nav.goToPose(make_pose(nav, r['x'], r['y'], r['yaw']))
    while not nav.isTaskComplete():
        rclpy.spin_once(nav, timeout_sec=0.2)
    return str(nav.getResult()) # SUCCEEDED / CANCELED / FAILED
```

4. Connect the camera to a VLM

Three of the four VLM services in Section V (OpenAI, Qwen, Ollama) speak the same OpenAI-compatible protocol, so one function covers them all and switching provider is one line. (Gemini uses its own google-genai SDK — see [18]; everything else in this example is unchanged.)

```
import base64, cv2
from openai import OpenAI

# OpenAI:      client = OpenAI()                # env: OPENAI_API_KEY
#             MODEL = 'gpt-5-mini' (or 'gpt-5-nano' / 'gpt-4o-mini')
# Qwen-VL (SG): client = OpenAI(api_key=KEY,
#             base_url='https://dashscope-intl.aliyuncs.com/compatible-mode/v1')
#             MODEL = 'qwen3-vl-flash' (or 'qwen3-vl-plus')
# Local Ollama: client = OpenAI(api_key='ollama',
#             base_url='http://localhost:11434/v1')
#             MODEL = 'qwen2.5vl:3b'

def ask_vlm(client, model, image_bgr, question):
    ok, buf = cv2.imencode('.jpg', image_bgr)
    b64 = base64.b64encode(buf).decode()
    resp = client.chat.completions.create(model=model, messages=[{
        'role': 'user',
        'content': [
            {'type': 'text', 'text': question},
            {'type': 'image_url',
             'image_url': {'url': f'data:image/jpeg;base64,{b64}'}}],
        # on OpenAI, add 'detail': 'low' next to 'url' to cut image cost ~10x
    ]})
    return resp.choices[0].message.content
```

Grabbing the latest camera frame in a ROS2 node:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge

class Camera(Node):
    def __init__(self):
        super().__init__('camera_buffer')
        self.bridge, self.frame = CvBridge(), None
        self.create_subscription(Image, '/camera/image_raw', self.cb, 10)
    def cb(self, msg):
        self.frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
```

A scene description is then one call:

```
report = ask_vlm(client, MODEL, cam.frame,
    'You are the eyes of an indoor robot. In 2-3 sentences, list the objects '
    'you can see and their locations. Be factual; do not guess objects that '
    'are not clearly visible.')
```

5. Visual grounding and object approach

For “move to the cup”, ask the VLM for a bounding box and convert it into velocity commands (a simple proportional controller). Qwen-VL models are trained for grounding and return reliable boxes; with GPT models (e.g., gpt-5-mini), ask for the box in the prompt and expect it to be approximate.

```
import json
from geometry_msgs.msg import Twist

GROUND = ('Locate the {obj} in the image. Answer ONLY with JSON: '
         '{"found": true/false, "bbox": [x1, y1, x2, y2]}'
         'in pixel coordinates of this image.')

def approach(cam, pub, obj):
    for _ in range(120):
        # safety-bounded loop
        ans = ask_vlm(client, MODEL, cam.frame, GROUND.format(obj=obj))
        d = json.loads(ans[ans.find('{'): ans.rfind('}') + 1])
        if not d['found']:
            spin_in_place(pub); continue # search behaviour
        x1, y1, x2, y2 = d['bbox']
        h, w = cam.frame.shape[:2]
        err = ((x1 + x2) / 2 - w / 2) / (w / 2) # -1 .. 1
        box_h = (y2 - y1) / h
        cmd = Twist()
        if box_h > 0.55: # close enough -> stop
            pub.publish(Twist()); return True
        cmd.angular.z = -0.5 * err
        cmd.linear.x = 0.12 if abs(err) < 0.25 else 0.0
        pub.publish(cmd)
    pub.publish(Twist()); return False
```

Tip: also subscribe to the laser scanner (/scan) and stop if the front distance drops below a threshold — the bounding-box size alone is not a safe stopping criterion for every object.

Coordinate conventions — the most common source of bugs: different models return bounding boxes in different conventions. Qwen3-VL (cloud API) returns coordinates normalized to 0–1000: convert with $x_{px} = x / 1000 \times \text{image width}$ (same for y with the height). The local Ollama qwen2.5vl returns absolute pixels on its internally resized input. GPT models return approximate boxes only — they are not trained for precise grounding, which is why Qwen is recommended for this task. With Qwen you can also use the official grounding prompt “Locate the <object>, output its bbox coordinates using JSON format”, which returns [{"bbox_2d": [x1,

y1, x2, y2], "label": ...}]]. Whatever model you use, draw the returned box on a saved frame first and check it before trusting it in the control loop.

6. The LLM manager

The manager is a text-only LLM call that turns each user utterance into one JSON command; your main loop dispatches it. Keep the chat history in the messages list so that follow-ups work:

```
SYSTEM = '''You are the dialogue manager of an indoor assistant robot.
The house has rooms numbered 1 to 4. For EVERY user message, reply ONLY with
one JSON object, no other text:
{"action": "goto_room", "room": <int 1-4>}
{"action": "describe"}
{"action": "approach", "object": "<object name>"}
{"action": "stop"}
{"action": "chat", "reply": "<answer or clarification question>"}
If the request is impossible (unknown room, unsafe), use "chat" to explain.'''

history = [{'role': 'system', 'content': SYSTEM}]
# TEXT_MODEL: any chat model, e.g. 'gpt-5-mini' (OpenAI) or 'qwen-plus' (Alibaba)
while True:
    user = input('You: ').strip()
    history.append({'role': 'user', 'content': user})
    out = client.chat.completions.create(model=TEXT_MODEL, messages=history)
    cmd = json.loads(out.choices[0].message.content)
    history.append({'role': 'assistant', 'content': json.dumps(cmd)})

    if cmd['action'] == 'goto_room':
        result = goto_room(cmd['room'])
        reply = report_arrival(result, cmd['room']) # + auto describe
    elif cmd['action'] == 'describe':
        reply = describe_scene()
    elif cmd['action'] == 'approach':
        ok = approach(cam, cmd_pub, cmd['object'])
        reply = f"I am now next to the {cmd['object']}." if ok else \
            f"Sorry, I could not find the {cmd['object']}."
    else:
        reply = cmd.get('reply', 'Stopped.')
    print('Robot:', reply)
    history.append({'role': 'assistant', 'content': reply})
```

Robustness hints: wrap `json.loads` in a `try/except` and re-ask the model once on a parse failure; cap the history length; log every VLM/LLM call (prompt, latency, tokens) — you will need these logs for the evaluation in Task 5.

7. Suggested timeline and vibe-coding tips

Week	Milestone
1	Environment set up; robot drives in the house world; camera visible (Task 1)

2	Map built; rooms numbered; goto_room works from the terminal chat (Task 2)
3	Scene description and visual Q&A working; VLM comparison started (Task 3)
4	Grounding + approach controller working (Task 4)
5	Integration; randomized evaluation runs (Task 5)
6	Demo video, report writing, code cleanup

Working with AI coding assistants: describe your ROS2 setup precisely (distro, robot model, topic names) in your prompts; paste real error messages instead of paraphrasing them; ask for small testable pieces (one node at a time) rather than the whole system; and always run and verify generated code before building on top of it. Keep a note of the prompts that worked — they go into your AI Usage Declaration.

VII. References

- [1] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” in Proc. NeurIPS, 2023. (LLaVA), <https://llava-vl.github.io>
- [2] S. Bai, et al., “Qwen2.5-VL technical report,” arXiv preprint arXiv:2502.13923, 2025; QwenLM, “Qwen3-VL cookbooks (spatial understanding / grounding),” <https://github.com/QwenLM/Qwen3-VL>
- [3] OpenAI, “GPT-4o system card,” 2024. <https://openai.com/index/gpt-4o-system-card/>
- [4] M. Ahn, et al., “Do as I can, not as I say: Grounding language in robotic affordances,” in Proc. CoRL, 2022. (SayCan)
- [5] J. Liang, et al., “Code as Policies: Language model programs for embodied control,” in Proc. IEEE ICRA, 2023.
- [6] A. Koubaa, “ROSGPT: Next-generation human-robot interaction with ChatGPT and ROS,” Preprints, 2023. <https://github.com/aniskoubaa/rosgpt>
- [7] NASA Jet Propulsion Laboratory, “ROSA: The Robot Operating System Agent,” <https://github.com/nasa-jpl/rosa>
- [8] ROBOTIS, “TurtleBot3 e-Manual,” <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/>